

دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۷

طراحی UART

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

تابستان ۱۴۰۵

۱.۷ مقدمه و اهداف

هدف این آزمایش، طراحی یک فرستنده‌گیرندهٔ ناهم‌گام همه‌منظوره[۱] است. مطابق دستورکار، بخش فرستنده هر بار یک کد هفت‌بیتی ASCII را به‌صورت سریال می‌فرستد: نخست یک بیت شروع[۲]، سپس یک بیت توازن[۳] و پس از آن هفت بیت داده؛ در پایان نیز دست‌کم یک بیت خاتمه[۴] ارسال می‌شود که در مجموع ده بیت می‌شود. بخش گیرنده پس از دریافت بیت شروع، هشت بیت مربوط به داده و توازن را به‌صورت سریال دریافت و در یک ثبات هشت‌بیتی ذخیره می‌کند. سرانجام باید با اتّصال دو واحد به یکدیگر نشان داده شود که مبادلهٔ داده به‌درستی انجام می‌شود.

نکتهٔ مهمی که پیش از هر چیز باید روشن شود، ترتیب غیرمتعارف بیت‌های قاب[۵] در این دستورکار است. در بیشتر پیاده‌سازی‌های متداول UART، بیت توازن *پس از* بیت‌های داده و بی‌درنگ پیش از بیت خاتمه فرستاده می‌شود؛ امّا دستورکار این آزمایش صراحتاً بیت توازن را *پیش از* بیت‌های داده قرار داده است. طرح حاضر دقیقاً از همین ترتیب خواسته‌شده پیروی می‌کند و این انتخاب در سراسر فرستنده، گیرنده و میز آزمون رعایت شده است.

روند انجام آزمایش

ابتدا ساختار قاب و ترتیب بیت‌ها از دستورکار استخراج و به‌صورت جدول درآمد. سپس فرستنده و گیرنده هر یک به‌صورت یک ماشین حالت متناهی[۶] جداگانه نوشته شدند و یک پیمانهٔ بالا این دو را با اتّصال حلقهٔ بازگشتی[۷] به هم بست. برای ارزیابی، یک میز آزمون[۸] خودآزما[۹] ساخته شد که افزون بر مبادلهٔ درست داده، توانایی گیرنده در تشخیص خطای توازن و خطای بیت خاتمه و نیز رفتار فرستنده در برابر درخواست هم‌زمان با اشغال بودن را می‌آزماید. پس از تأیید درستی در ModelSim، همان سه پروندهٔ سنتزپذیر در Quartus II همگردانی شدند و نمای سطح انتقال ثبات‌ها[۱۰]، نماهای فناوری[۱۱]، گزارش مصرف منابع و تحلیل زمانی طرح بررسی شد.

[1] Universal Asynchronous Receiver Transmitter (UART)
[2] Start Bit
[3] Parity Bit
[4] Stop Bit
[5] Frame
[6] Finite State Machine
[7] Loopback
[8] Testbench
[9] Self-Checking Testbench
[10] Register Transfer Level (RTL) View
[11] Technology Map Viewer

۲.۷ ساختار قاب و تصمیم‌های طراحی

الف) ترتیب بیت‌ها

جدول زیر ده بیت قاب را به همان ترتیبی که روی خط ظاهر می‌شوند نشان می‌دهد.

شماره	بیت	مقدار
1	شروع	همواره صفر
2	توازن	توازن زوج هفت بیت داده
3	دادهٔ D_0	کم‌ارزش‌ترین بیت
4	دادهٔ D_1	—
5	دادهٔ D_2	—
6	دادهٔ D_3	—
7	دادهٔ D_4	—
8	دادهٔ D_5	—
9	دادهٔ D_6	پرارزش‌ترین بیت
10	خاتمه	همواره یک

جدول ۱: ترتیب ده بیت قاب مطابق دستورکار

بیت‌های داده از کم‌ارزش به پرارزش فرستاده می‌شوند، که قرارداد متعارف UART است. بیت توازن نیز از نوع زوج[۱۲] است و در کد با عملگر کاهشی یای‌انحصاری روی هفت بیت داده ساخته می‌شود؛ یعنی اگر شمار یک‌های داده زوج باشد بیت توازن صفر و در غیر این صورت یک می‌شود.

ب) خط بی‌کار و آشکارسازی شروع

خط ارسال در حالت بی‌کار روی یک نگه داشته می‌شود. چون بیت شروع همواره صفر است، گیرنده تنها با دیدن یک سطح پایین روی خط بی‌کارِ یک، آغاز قاب را تشخیص می‌دهد. این همان سازوکار متعارف UART است و به همین دلیل نیازی به هیچ سیگنال هم‌گام‌سازی جداگانه‌ای میان فرستنده و گیرنده نیست؛ صفت «ناهم‌گام» در نام این واحد نیز به همین ویژگی اشاره دارد.

[12] Even Parity

ج) طول هر بیت و پارامتر TICKS_BIT

در یک UART واقعی، طول هر بیت روی خط از نرخ باود[۱۳] تعیین می‌شود و معمولاً بسیار بزرگ‌تر از یک پالس ساعت است. در این طرح، طول هر بیت با پارامتر TICKS_BIT بر حسب شمار پالس‌های ساعت بیان شده است و مقدار پیش‌فرض آن دو است. با این کار، هم شبیه‌سازی در زمان معقول اجرا می‌شود و هم طرح به یک نرخ باود مشخّص وابسته نمی‌ماند؛ برای رسیدن به نرخ باود دلخواه کافی است این پارامتر برابر نسبت بسامد ساعت به نرخ باود گرفته شود.

هر دو پیمانهٔ فرستنده و گیرنده این پارامتر را می‌پذیرند و پیمانهٔ بالا آن را به هر دو منتقل می‌کند، بنابراین دو سوی خط همواره با یک طول بیت یکسان کار می‌کنند.

د) وارسی درستی قاب در گیرنده

دستورکار تنها ذخیرهٔ داده و توازن در یک ثبات هشت‌بیتی را خواسته است. در این طرح، افزون بر ذخیرهٔ داده، خروجی سومی نیز به نام data_correct افزوده شد که سلامت قاب دریافتی را گزارش می‌کند. این خروجی تنها هنگامی یک می‌شود که هر دو شرط زیر برقرار باشند: بیت خاتمه در جای خود برابر یک باشد و توازن محاسبه‌شده از روی هفت بیت داده با بیت توازن دریافتی بخواند. به این ترتیب، هم خطای تک‌بیتی در داده و هم از دست رفتن هم‌گامی قاب آشکار می‌شود.

۳.۷ توصیف فرستنده

فرستنده یک ماشین حالت با پنج حالت است: بی‌کار، شروع، توازن، داده و خاتمه. در حالت بی‌کار، خط روی یک نگه داشته می‌شود و پرچم اشغال صفر است. با یک شدن data_new، هفت بیت داده در یک ثبات درونی نگه داشته می‌شود، بیت توازن همان‌جا محاسبه و ذخیره می‌گردد، خط به صفر می‌رود و ماشین وارد حالت شروع می‌شود.

نکتهٔ درخور توجّه آن است که درخواست ارسال تنها در حالت بی‌کار پذیرفته می‌شود. اگر در میانهٔ ارسال یک قاب، درخواست تازه‌ای برسد، آن درخواست به‌سادگی نادیده گرفته می‌شود و قاب در حال ارسال آسیب نمی‌بیند. این تصمیم در بخش شبیه‌سازی به‌صورت جداگانه آزموده شده است.

در هر یک از حالت‌های بعدی، یک شمارنده به اندازهٔ TICKS_BIT پالس ساعت می‌شمارد و سپس بیت بعدی را روی خط می‌گذارد. در حالت داده، شمارندهٔ bit_index از صفر تا شش پیش می‌رود و پس از فرستادن D_6، خط به یک بازمی‌گردد و ماشین وارد حالت خاتمه می‌شود. پرچم اشغال تا پایان بیت خاتمه

[13] Baud Rate

بالا می‌ماند تا میز آزمون یا هر مصرف‌کنندهٔ دیگری بداند که فرستنده هنوز آزاد نیست.

```
`timescale 1ns/1ps

module UARTSender #(
    parameter integer TICKS_BIT = 2
)(
    input  wire       clk,
    input  wire       rstN,
    input  wire       data_new,
    input  wire [6:0] data_send,
    output reg        tx,
    output reg        busy
);

    localparam [2:0] S_IDLE   = 3'd0;
    localparam [2:0] S_START  = 3'd1;
    localparam [2:0] S_PARITY = 3'd2;
    localparam [2:0] S_DATA   = 3'd3;
    localparam [2:0] S_STOP   = 3'd4;

    reg [2:0] state;
    reg [2:0] bit_index;
    reg [6:0] data_reg;
    reg       parity_reg;
    integer   tick_count;

    always @(posedge clk or negedge rstN) begin
        if (!rstN) begin
            state      <= S_IDLE;
            bit_index  <= 3'd0;
            data_reg   <= 7'd0;
            parity_reg <= 1'b0;
            tick_count <= 0;
            tx         <= 1'b1;
            busy       <= 1'b0;
        end else begin
            case (state)
                S_IDLE: begin
                    tx         <= 1'b1;
                    busy       <= 1'b0;
                    tick_count <= 0;
                    bit_index  <= 3'd0;

                    // A new request is accepted only in IDLE.
                    if (data_new) begin
                        data_reg   <= data_send;
                        parity_reg <= ^data_send;
                        tx         <= 1'b0;
                        busy       <= 1'b1;
                        state      <= S_START;
                    end
                end

                S_START: begin
                    busy <= 1'b1;
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count <= 0;
                        tx         <= parity_reg;
                        state      <= S_PARITY;
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                S_PARITY: begin
                    busy <= 1'b1;
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count <= 0;
                        bit_index  <= 3'd0;
                        tx         <= data_reg[0];
```

```
                        state      <= S_DATA;
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                S_DATA: begin
                    busy <= 1'b1;
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count <= 0;

                        // Data bits are transmitted LSB first: D0 through D6.
                        if (bit_index == 3'd6) begin
                            tx    <= 1'b1;
                            state <= S_STOP;
                        end else begin
                            bit_index <= bit_index + 1'b1;
                            tx        <= data_reg[bit_index + 1'b1];
                        end
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                S_STOP: begin
                    busy <= 1'b1;
                    tx   <= 1'b1;
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count <= 0;
                        busy       <= 1'b0;
                        state      <= S_IDLE;
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                default: begin
                    state      <= S_IDLE;
                    tick_count <= 0;
                    bit_index  <= 0;
                    tx         <= 1'b1;
                    busy       <= 1'b0;
                end
            endcase
        end
    end

endmodule
```

کد ۱: فرستنده‌ی UART

۴.۷ توصیف گیرنده

گیرنده چهار حالت دارد: بی‌کار، شروع، داده و خاتمه. در حالت بی‌کار، تنها پایین رفتن خط دنبال می‌شود. پس از آشکارسازی شروع، ماشین به اندازهٔ یک طول بیت صبر می‌کند و سپس بیت توازن را نمونه‌برداری می‌کند؛ پس از آن هفت بیت داده یکی‌یکی در ثبات درونی نوشته می‌شوند و سرانجام در حالت خاتمه، بیت خاتمه بررسی می‌گردد.

زمان‌بندی نمونه‌برداری در اینجا اهمّیّت دارد. چون شمارش هر بیت از لحظهٔ آشکارسازی لبهٔ شروع آغاز می‌شود و نمونه‌برداری در پایان شمارش انجام می‌گیرد، هر بیت در میانهٔ بازهٔ خود و نه در لبه‌های آن خوانده

می‌شود. همین فاصله از لبه‌هاست که خواندن را در برابر اختلاف جزئی زمان‌بندی میان دو سوی خط مقاوم می‌کند.

خروجی data_new_rec تنها برای یک پالس ساعت بالا می‌رود و پایان یک قاب کامل را اعلام می‌کند؛ حال آنکه data_rec و data_correct تا رسیدن قاب بعدی مقدار خود را نگه می‌دارند. این تفکیک باعث می‌شود مصرف‌کنندهٔ خروجی بتواند با یک پالس، لحظهٔ آمادگی دادهٔ تازه را تشخیص دهد و در عین حال داده تا استفادهٔ بعدی پایدار بماند.

```
`timescale 1ns/1ps

module UARTReceiver #(
    parameter integer TICKS_BIT = 2
)(
    input  wire       clk,
    input  wire       rstN,
    input  wire       rx,
    output reg  [6:0] data_rec,
    output reg        data_new_rec,
    output reg        data_correct
);

    localparam [2:0] S_IDLE  = 3'd0;
    localparam [2:0] S_START = 3'd1;
    localparam [2:0] S_DATA  = 3'd2;
    localparam [2:0] S_STOP  = 3'd3;

    reg [2:0] state;
    reg [2:0] bit_index;
    reg [6:0] data_reg;
    reg       parity_received;
    integer   tick_count;

    always @(posedge clk or negedge rstN) begin
        if (!rstN) begin
            state           <= S_IDLE;
            bit_index       <= 3'd0;
            data_reg        <= 7'd0;
            parity_received <= 1'b0;
            tick_count      <= 0;
            data_rec        <= 7'd0;
            data_new_rec    <= 1'b0;
            data_correct    <= 1'b0;
        end else begin
            // Asserted for one clock when a complete frame is received.
            data_new_rec <= 1'b0;

            case (state)
                S_IDLE: begin
                    tick_count <= 0;
                    bit_index  <= 3'd0;

                    // A low level on the idle-high line indicates the start bit.
                    if (rx == 1'b0) begin
                        state      <= S_START;
                        tick_count <= 0;
                    end
                end

                S_START: begin
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count      <= 0;
                        parity_received <= rx;
                        bit_index       <= 3'd0;
                        state           <= S_DATA;
                    end else begin
```

```
                        tick_count <= tick_count + 1;
                    end
                end

                S_DATA: begin
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count          <= 0;
                        data_reg[bit_index] <= rx;

                        if (bit_index == 3'd6) begin
                            state <= S_STOP;
                        end else begin
                            bit_index <= bit_index + 1'b1;
                        end
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                S_STOP: begin
                    if (tick_count == TICKS_BIT - 1) begin
                        tick_count   <= 0;
                        data_rec     <= data_reg;
                        data_new_rec <= 1'b1;

                        // A frame is valid only if stop=1 and parity matches the data.
                        if ((rx == 1'b1) && ((^data_reg) == parity_received))
                            data_correct <= 1'b1;
                        else
                            data_correct <= 1'b0;

                        state <= S_IDLE;
                    end else begin
                        tick_count <= tick_count + 1;
                    end
                end

                default: begin
                    state           <= S_IDLE;
                    bit_index       <= 0;
                    tick_count      <= 0;
                    data_new_rec    <= 1'b0;
                    data_correct    <= 1'b0;
                end
            endcase
        end
    end

endmodule
```

کد ۲: گیرنده‌ی UART

۵.۷ اتّصال دو واحد

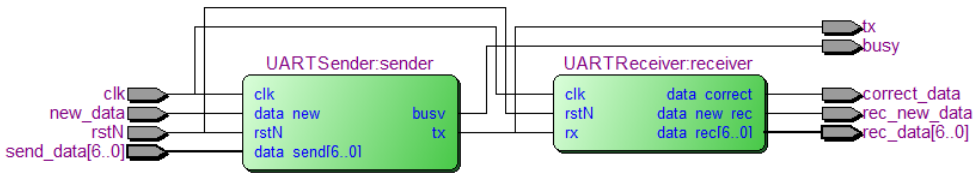
دستورکار خواسته است که با اتّصال دو واحد به یکدیگر، درستی مبادلهٔ داده نشان داده شود. پیمانهٔ بالا دقیقاً همین کار را انجام می‌دهد: یک نمونه از فرستنده و یک نمونه از گیرنده می‌سازد و خروجی tx فرستنده را مستقیماً به ورودی rx گیرنده می‌بندد. این همان آرایش حلقهٔ بازگشتی است و در آن، هر قابی که فرستنده روی خط می‌گذارد بی‌درنگ توسط گیرنده خوانده و بازسازی می‌شود.

```
`timescale 1ns/1ps
```

```
2
3  module UARTTop #(
4      parameter integer TICKS_BIT = 2
5  )(
6      input  wire       clk,
7      input  wire       rstN,
8      input  wire       new_data,
9      input  wire [6:0] send_data,
10     output wire       tx,
11     output wire       busy,
12     output wire [6:0] rec_data,
13     output wire       rec_new_data,
14     output wire       correct_data
15 );
16
17     UARTSender #(
18         .TICKS_BIT(TICKS_BIT)
19     ) sender (
20         .clk       (clk),
21         .rstN      (rstN),
22         .data_new  (new_data),
23         .data_send (send_data),
24         .tx        (tx),
25         .busy      (busy)
26     );
27
28     // Internal loopback connects the transmitter output directly to the receiver.
29     UARTReceiver #(
30         .TICKS_BIT(TICKS_BIT)
31     ) receiver (
32         .clk          (clk),
33         .rstN         (rstN),
34         .rx           (tx),
35         .data_rec     (rec_data),
36         .data_new_rec (rec_new_data),
37         .data_correct (correct_data)
38     );
39
40 endmodule
```

کد ۳: پیمانه‌ی بالا و اتّصال حلقه‌ی بازگشتی فرستنده به گیرنده

نمای سطح انتقال ثبات‌های پیمانهٔ بالا همین ساختار را نشان می‌دهد: دو بلوک مجزّای فرستنده و گیرنده که تنها پیوند میانشان، خط سریال است.



شکل ۱: نمای سطح انتقال ثبات‌های پیمانه‌ی بالا و مسیر حلقه‌ی بازگشتی

۶.۷ میز آزمون

میز آزمون افزون بر پیمانهٔ بالا، یک گیرندهٔ مستقل دیگر نیز می‌سازد که ورودی سریال آن مستقیماً از میز آزمون تغذیه می‌شود. هدف از این گیرندهٔ دوم، تزریق قاب‌های معیوب است: چون فرستندهٔ سالم هرگز قاب نادرست تولید نمی‌کند، برای آزمودن توانایی تشخیص خطا باید قاب را دستی و بیت‌به‌بیت ساخت. وظیفهٔ send_manual_frame همین کار را می‌کند و دو ورودی جداگانه دارد که با آن‌ها می‌توان بیت توازن یا بیت خاتمه را عمداً خراب کرد.

آزمون از پنج مرحله تشکیل شده است:

- **مرحلهٔ ۱ — قاب سالم**: یک کد هفت‌بیتی از مسیر حلقهٔ بازگشتی فرستاده و بررسی می‌شود که همان مقدار در گیرنده بازسازی شود و پرچم درستی بالا برود.
- **مرحلهٔ ۲ — خطای توازن**: قابی با بیت توازن وارونه ساخته می‌شود و بررسی می‌گردد که گیرنده آن را نادرست اعلام کند.
- **مرحلهٔ ۳ — خطای بیت خاتمه**: قابی با بیت خاتمهٔ صفر ساخته می‌شود و انتظار می‌رود پرچم درستی پایین بماند.
- **مرحلهٔ ۴ — دو قاب پیاپی**: دو کد متفاوت پشت سر هم فرستاده می‌شوند تا نشان داده شود ماشین پس از هر قاب به‌درستی به حالت بی‌کار بازمی‌گردد.
- **مرحلهٔ ۵ — درخواست هم‌زمان با اشغال**: در میانهٔ ارسال یک قاب، درخواست تازه‌ای اعمال می‌شود. آزمون هم بررسی می‌کند که قاب در حال ارسال دست‌نخورده بماند و هم اینکه از درخواست نادیده‌گرفته‌شده هیچ قاب اضافه‌ای ساخته نشود.

مرحلهٔ پنجم دو وارسی جداگانه دارد و همین آن را از یک آزمون سطحی متمایز می‌کند. وارسی نخست مقدار بازسازی‌شده را می‌سنجد و وارسی دوم، پس از آزاد شدن فرستنده، به مدّت ده طول بیت دیگر منتظر می‌ماند تا مطمئن شود هیچ پالس data_new_rec تازه‌ای ظاهر نمی‌شود.

```
`timescale 1ns/1ps

module Tester;

    parameter integer TICKS_BIT = 2;

    reg        clk;
    reg        rstN;
    reg        new_data;
    reg  [6:0] send_data;

    wire       tx;
    wire       busy;
    wire [6:0] rec_data;
    wire       rec_new_data;
```

```
    wire       correct_data;

    // Independent RX path used to inject parity and stop-bit errors.
    reg        fault_rx;
    wire [6:0] fault_data;
    wire       fault_new_data;
    wire       fault_correct;

    integer errors;

    UARTop #(
        .TICKS_BIT(TICKS_BIT)
    ) dut (
        .clk          (clk),
        .rstN         (rstN),
        .new_data     (new_data),
        .send_data    (send_data),
        .tx           (tx),
        .busy         (busy),
        .rec_data     (rec_data),
        .rec_new_data (rec_new_data),
        .correct_data (correct_data)
    );

    UARTReceiver #(
        .TICKS_BIT(TICKS_BIT)
    ) fault_receiver (
        .clk          (clk),
        .rstN         (rstN),
        .rx           (fault_rx),
        .data_rec     (fault_data),
        .data_new_rec (fault_new_data),
        .data_correct (fault_correct)
    );

    // 10 ns clock period.
    always #5 clk = ~clk;

    task reset_all;
        begin
            rstN      = 1'b0;
            new_data  = 1'b0;
            send_data = 7'd0;
            fault_rx  = 1'b1;
            repeat (4) @(posedge clk);
            @(negedge clk);
            rstN = 1'b1;
            repeat (2) @(posedge clk);
        end
    endtask

    task send_valid_word;
        input [6:0] value;
        begin
            wait (busy == 1'b0);
            @(negedge clk);
            send_data = value;
            new_data  = 1'b1;
            @(negedge clk);
            new_data  = 1'b0;
        end
    endtask

    // Hold each manually driven serial bit for TICKS_BIT clock cycles.
    task drive_fault_bit;
        input bit_value;
        begin
            @(negedge clk);
            fault_rx = bit_value;
            repeat (TICKS_BIT) @(posedge clk);
        end
    endtask
```

```
88
89      // Generate a frame with optional parity or stop-bit corruption.
90      task send_manual_frame;
91          input [6:0] value;
92          input       parity_error;
93          input       stop_error;
94          integer i;
95          reg p;
96          begin
97              p = (^value) ^ parity_error;
98
99              drive_fault_bit(1'b1);
100             drive_fault_bit(1'b0);
101             drive_fault_bit(p);
102
103             for (i = 0; i < 7; i = i + 1)
104                 drive_fault_bit(value[i]);
105
106             drive_fault_bit(stop_error ? 1'b0 : 1'b1);
107             drive_fault_bit(1'b1);
108         end
109     endtask
110
111     initial begin
112         clk    = 1'b0;
113         errors = 0;
114         reset_all;
115
116         $display("================================================");
117         $display("UART Lab 7 simulation started");
118         $display("TICKS_BIT = %0d", TICKS_BIT);
119         $display("================================================");
120
121         // Test 1: valid frame.
122         $display("\nTEST 1 - correct frame: 7'b1011001");
123         send_valid_word(7'b1011001);
124         @(posedge rec_new_data);
125         #1;
126         if ((rec_data == 7'b1011001) && (correct_data == 1'b1))
127             $display("PASS: rec_data=%b correct_data=%b", rec_data, correct_data);
128         else begin
129             $display("FAIL: rec_data=%b correct_data=%b", rec_data, correct_data);
130             errors = errors + 1;
131         end
132
133         // Test 2: corrupted parity bit.
134         $display("\nTEST 2 - intentional parity error");
135         fork
136             send_manual_frame(7'b1011001, 1'b1, 1'b0);
137             begin
138                 @(posedge fault_new_data);
139                 #1;
140                 if (fault_correct == 1'b0)
141                     $display("PASS: parity error detected, correct_data=0");
142                 else begin
143                     $display("FAIL: parity error was not detected");
144                     errors = errors + 1;
145                 end
146             end
147         join
148
149         // Test 3: stop bit forced to zero.
150         $display("\nTEST 3 - intentional stop-bit error");
151         fork
152             send_manual_frame(7'b1011001, 1'b0, 1'b1);
153             begin
154                 @(posedge fault_new_data);
155                 #1;
156                 if (fault_correct == 1'b0)
157                     $display("PASS: stop-bit error detected, correct_data=0");
158                 else begin
159                     $display("FAIL: stop-bit error was not detected");
```

```
                    errors = errors + 1;
                end
            end
        join

        reset_all;

        // Test 4: two valid frames sent sequentially.
        $display("\nTEST 4 - two sequential frames");

        send_valid_word(7'b1011001);
        @(posedge rec_new_data);
        #1;
        if ((rec_data == 7'b1011001) && (correct_data == 1'b1))
            $display("PASS frame A: %b", rec_data);
        else begin
            $display("FAIL frame A: rec_data=%b correct=%b", rec_data, correct_data);
            errors = errors + 1;
        end

        send_valid_word(7'b0101101);
        @(posedge rec_new_data);
        #1;
        if ((rec_data == 7'b0101101) && (correct_data == 1'b1))
            $display("PASS frame B: %b", rec_data);
        else begin
            $display("FAIL frame B: rec_data=%b correct=%b", rec_data, correct_data);
            errors = errors + 1;
        end

        // Test 5: data_new during busy must be ignored.
        $display("\nTEST 5 - data_new asserted while busy");

        send_valid_word(7'b1100101);
        wait (busy == 1'b1);
        repeat (3) @(posedge clk);

        @(negedge clk);
        if (busy !== 1'b1) begin
            $display("FAIL: transmitter was expected to be busy");
            errors = errors + 1;
        end

        send_data = 7'b0011110;
        new_data  = 1'b1;
        @(negedge clk);
        new_data  = 1'b0;

        // The active frame must retain the data latched before busy was asserted.
        @(posedge rec_new_data);
        #1;
        if ((rec_data == 7'b1100101) && (correct_data == 1'b1))
            $display("PASS: active frame was not corrupted; rec_data=%b", rec_data);
        else begin
            $display("FAIL: busy-time data_new corrupted active frame: rec_data=%b correct=%b",
                     rec_data, correct_data);
            errors = errors + 1;
        end

        // Verify that the ignored request does not create an extra frame.
        wait (busy == 1'b0);
        repeat (10*TICKS_BIT + 5) begin
            @(posedge clk);
            #1;
            if (rec_new_data === 1'b1) begin
                $display("FAIL: a second frame was generated from data_new asserted during busy");
                errors = errors + 1;
            end
        end

        if (rec_data == 7'b1100101)
            $display("PASS: busy-time request was ignored; no extra frame was received");
```

```
232
233         $display("\n===========================================");
234         if (errors == 0)
235             $display("ALL REQUIRED + EXTRA TESTS PASSED");
236         else
237             $display("SIMULATION FINISHED WITH %0d ERROR(S)", errors);
238         $display("===========================================");
239
240         #20;
241         $finish;
242     end
243
244     initial begin
245         $monitor("time=%0t rstN=%b new_data=%b send_data=%b tx=%b busy=%b rec_data=%b rec_new=%b
     correct=%b",
246                  $time, rstN, new_data, send_data, tx, busy, rec_data, rec_new_data, correct_data);
247     end
248
249 endmodule
```

کد ۴: میز آزمون خودآزمای UART

۷.۷ اجرا در ModelSim

اجرای شبیه‌سازی با یک پروندهٔ فرمان انجام می‌شود که کتابخانه را از نو می‌سازد، چهار پرونده را همگردانی می‌کند، سیگنال‌ها را در دو دستهٔ حلقهٔ بازگشتی و گیرندهٔ آزمون خطا به پنجرهٔ موج می‌افزاید و شبیه‌سازی را تا پایان اجرا می‌کند.

```
transcript on

if {[file exists work]} {
    vdel -lib work -all
}
vlib work
vmap work work

vlog UARTSender.v UARTReceiver.v UARTTop.v Tester.v
vsim work.Tester

view wave
add wave -divider "TOP LOOPBACK"
add wave sim:/Tester/clk
add wave sim:/Tester/rstN
add wave sim:/Tester/new_data
add wave -radix binary sim:/Tester/send_data
add wave sim:/Tester/tx
add wave sim:/Tester/busy
add wave -radix binary sim:/Tester/rec_data
add wave sim:/Tester/rec_new_data
add wave sim:/Tester/correct_data

add wave -divider "FAULT TEST RECEIVER"
add wave sim:/Tester/fault_rx
add wave -radix binary sim:/Tester/fault_data
add wave sim:/Tester/fault_new_data
add wave sim:/Tester/fault_correct

run -all
wave zoom full
```

کد ۵: پرونده‌ی فرمان اجرای شبیه‌سازی

خروجی متنی آزمون نشان می‌دهد که هر پنج مرحله با موفّقیّت گذشته‌اند. تصویرهای زیر رونوشت کامل اجرا را از آغاز تا پیام پایانی دنبال می‌کنند.

```
# run -all
# time=0 rstN=0 new_data=0 send_data=0000000 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
# time=40000 rstN=1 new_data=0 send_data=0000000 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
# ============================================
# UART Lab 7 simulation started
# TICKS_BIT = 2
# ============================================
#
# TEST 1 - correct frame: 7'b1011001
# time=60000 rstN=1 new_data=1 send_data=1011001 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
# time=65000 rstN=1 new_data=1 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=70000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=105000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=125000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=165000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=205000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=225000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=255000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=1011001 rec_new=1 correct=1
# PASS: rec_data=1011001 correct_data=1
#
# TEST 2 - intentional parity error
# time=265000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=0 rec_data=1011001 rec_new=0 correct=1
# PASS: parity error detected, correct_data=0
#
# TEST 3 - intentional stop-bit error
# PASS: stop-bit error detected, correct_data=0
# time=735000 rstN=0 new_data=0 send_data=0000000 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
# time=780000 rstN=1 new_data=0 send_data=0000000 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
#
# TEST 4 - two sequential frames
# time=800000 rstN=1 new_data=1 send_data=1011001 tx=1 busy=0 rec_data=0000000 rec_new=0 correct=0
# time=805000 rstN=1 new_data=1 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=810000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=845000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=865000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
```

شکل ۲: رونوشت اجرا؛ مرحله‌های یک تا سه و آغاز مرحله‌ی چهارم

```
# time=905000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=945000 rstN=1 new_data=0 send_data=1011001 tx=0 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=965000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=0000000 rec_new=0 correct=0
# time=995000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=1 rec_data=1011001 rec_new=1 correct=1
# PASS frame A: 1011001
# time=1005000 rstN=1 new_data=0 send_data=1011001 tx=1 busy=0 rec_data=1011001 rec_new=0 correct=1
# time=1010000 rstN=1 new_data=1 send_data=0101101 tx=1 busy=0 rec_data=1011001 rec_new=0 correct=1
# time=1015000 rstN=1 new_data=1 send_data=0101101 tx=0 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1020000 rstN=1 new_data=0 send_data=0101101 tx=0 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1055000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1075000 rstN=1 new_data=0 send_data=0101101 tx=0 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1095000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1135000 rstN=1 new_data=0 send_data=0101101 tx=0 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1155000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1175000 rstN=1 new_data=0 send_data=0101101 tx=0 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1195000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=1 rec_data=1011001 rec_new=0 correct=1
# time=1205000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=1 rec_data=0101101 rec_new=1 correct=1
# PASS frame B: 0101101
#
# TEST 5 - data_new asserted while busy
# time=1215000 rstN=1 new_data=0 send_data=0101101 tx=1 busy=0 rec_data=0101101 rec_new=0 correct=1
# time=1220000 rstN=1 new_data=1 send_data=1100101 tx=1 busy=0 rec_data=0101101 rec_new=0 correct=1
# time=1225000 rstN=1 new_data=1 send_data=1100101 tx=0 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1230000 rstN=1 new_data=0 send_data=1100101 tx=0 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1260000 rstN=1 new_data=1 send_data=0011110 tx=0 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1265000 rstN=1 new_data=1 send_data=0011110 tx=1 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1270000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1285000 rstN=1 new_data=0 send_data=0011110 tx=0 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1305000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1325000 rstN=1 new_data=0 send_data=0011110 tx=0 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1365000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=1 rec_data=0101101 rec_new=0 correct=1
# time=1415000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=1 rec_data=1100101 rec_new=1 correct=1
# PASS: active frame was not corrupted; rec_data=1100101
# time=1425000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=0 rec_data=1100101 rec_new=0 correct=1
```

شکل ۳: ادامه‌ی رونوشت؛ مرحله‌های چهارم و پنجم

```
# time=1415000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=1 rec_data=1100101 rec_new=1 correct=1
# PASS: active frame was not corrupted; rec_data=1100101
# time=1425000 rstN=1 new_data=0 send_data=0011110 tx=1 busy=0 rec_data=1100101 rec_new=0 correct=1
# PASS: busy-time request was ignored; no extra frame was received
#
# ============================================
# ALL REQUIRED + EXTRA TESTS PASSED
# ============================================
# ** Note: $finish    : Tester.v(241)
#    Time: 1696 ns  Iteration: 0  Instance: /Tester
# 1
# Break in Module Tester at Tester.v line 241
# Simulation Breakpoint: 1
# Break in Module Tester at Tester.v line 241
# MACRO ./run.do PAUSED at line 30

VSIM(paused)>
```

شکل ۴: پیام پایانی و گزارش قبولی همه‌ی آزمون‌ها

آخرین سطرها نشان می‌دهند که شمارندهٔ خطا صفر مانده و شبیه‌سازی در لحظهٔ ۱۶۹۶ نانوثانیه به پایان رسیده است.

۸.۷ تحلیل شکل موج‌ها

الف) کلید خواندن شکل موج

پیش از هر تحلیلی باید دو عدد را در دست داشت، چون همهٔ بازه‌های زمانی این آزمایش از آن‌ها به دست می‌آیند. نخست، دورهٔ ساعت میز آزمون ۱۰ نانوثانیه است. دوم، پارامتر TICKS_BIT برابر دو است،

یعنی هر بیت روی خط به اندازهٔ دو پالس ساعت پایدار می‌ماند. از ضرب این دو نتیجه می‌شود:

$$\text{طول هر بیت} = 2 \times 10 = 20 \text{ نانوثانیه}, \qquad \text{طول یک قاب کامل} = 10 \times 20 = 200 \text{ نانوثانیه}.$$

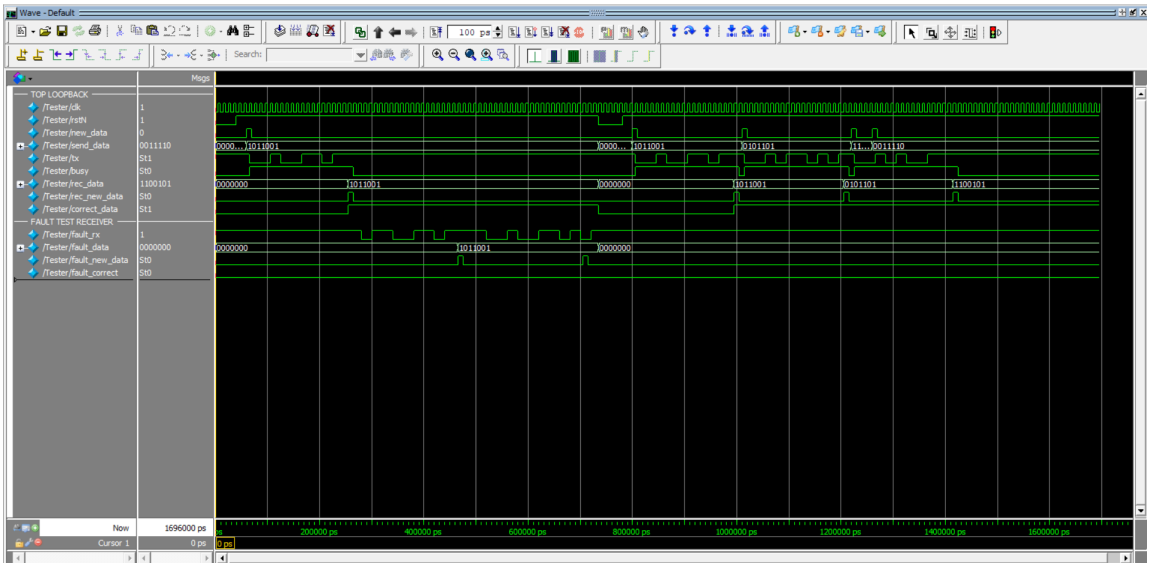
همین دو عدد، نکتهٔ کلیدی ظاهر شکل موج را نیز توضیح می‌دهند. در نگاه نخست ممکن است پرسیده شود چرا پالس‌های روی خط tx پهنای یکسانی ندارند و برخی دو برابر برخی دیگرند. پاسخ این است که خط tx هیچ نشانه‌ای برای مرز میان بیت‌ها ندارد؛ اگر دو بیت پیاپی مقدار یکسانی داشته باشند، روی خط به یک پالس پیوستهٔ ۴۰ نانوثانیه‌ای دیده می‌شوند. بنابراین پهنای هر پالس همواره مضرب صحیحی از ۲۰ نانوثانیه است و شمار آن مضرب، تعداد بیت‌های هم‌مقدار پشت سر هم را می‌گوید. همین ویژگی است که ایجاب می‌کند گیرنده به‌جای شمردن لبه‌ها، از لحظهٔ آشکارسازی شروع، زمان را خودش بشمارد.

ب) سیگنال‌های پنجره‌ی موج

جدول زیر سیگنال‌های نمایش‌داده‌شده در پنجرهٔ موج و معنای آن‌ها را گردآوری می‌کند.

دسته	سیگنال	معنا
حلقهٔ بازگشتی	clk	ساعت با دورهٔ ۱۰ نانوثانیه
حلقهٔ بازگشتی	rstN	بازنشانی ناهم‌زمان فعّال‌صفر
حلقهٔ بازگشتی	new_data	درخواست ارسال یک کد تازه
حلقهٔ بازگشتی	send_data	کد هفت‌بیتی ورودی فرستنده
حلقهٔ بازگشتی	tx	خط سریال خروجی فرستنده
حلقهٔ بازگشتی	busy	اشغال بودن فرستنده
حلقهٔ بازگشتی	rec_data	کد بازسازی‌شده در گیرنده
حلقهٔ بازگشتی	rec_new_data	پالس یک‌کلاکی پایان قاب
حلقهٔ بازگشتی	correct_data	سلامت قاب دریافتی
آزمون خطا	fault_rx	خط سریالی که میز آزمون دستی می‌راند
آزمون خطا	fault_data	دادهٔ گیرنده‌ی آزمون خطا
آزمون خطا	fault_new_data	پالس پایان قاب معیوب
آزمون خطا	fault_correct	نتیجه‌ی وارسی قاب معیوب

جدول ۲: سیگنال‌های پنجره‌ی موج و معنای آن‌ها



شکل ۵: شکل موج کامل شبیه‌سازی و هر پنج مرحله‌ی آزمون

در نمای کامل، پنج بازهٔ فعّالیّت از هم جدا دیده می‌شوند که به‌ترتیب به پنج مرحلهٔ آزمون مربوط‌اند. جدول زیر این بازه‌ها را مشخّص می‌کند تا در ادامه بتوان به هر یک ارجاع داد.

مرحله	بازه (نانوثانیه)	آنچه در آن رخ می‌دهد
1	65—265	قاب سالم روی مسیر حلقهٔ بازگشتی
2	280—480	قاب دستی با بیت توازن وارونه
3	520—720	قاب دستی با بیت خاتمهٔ صفر
4	805—1215	دو قاب سالم پیاپی
5	1225—1425	قاب سالم به‌همراه درخواست نادیده‌گرفته‌شده

جدول ۳: بازه‌های زمانی پنج مرحله‌ی آزمون در شکل موج کامل

ج) تحلیل بازه‌به‌بازه‌ی قاب سالم

مرحلهٔ نخست، کد 1011001_2 را می‌فرستد. برای فهمیدن اینکه چرا خط tx دقیقاً همان شکلی می‌شود که در نما دیده می‌شود، ابتدا باید بیت‌های این کد را جدا کرد. بیت‌ها از کم‌ارزش به پرارزش عبارت‌اند از

$$D_0 = 1,\ D_1 = 0,\ D_2 = 0,\ D_3 = 1,\ D_4 = 1,\ D_5 = 0,\ D_6 = 1,$$

یعنی چهار بیت یک و سه بیت صفر. چون شمار یک‌ها زوج است، بیت توازن زوج برابر صفر می‌شود:

$$P = D_0 \oplus D_1 \oplus \cdots \oplus D_6 = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 0.$$

درخواست ارسال در لحظهٔ ۶۰ نانوثانیه اعمال می‌شود و فرستنده آن را در نخستین لبهٔ فعّال پس از آن، یعنی

۶۵ نانوثانیه، می‌پذیرد. از همین‌جا قاب آغاز می‌شود. جدول زیر هر ده بیت قاب را با بازهٔ زمانی دقیق و دلیل مقدار آن می‌آورد.

بازه (نانوثانیه)	بیت	tx	چرا این مقدار
65–85	شروع	0	قرارداد قاب؛ پایین رفتن خط بی‌کار، آغاز را اعلام می‌کند
85–105	توازن	0	شمار یک‌های داده زوج است
105–125	D_0	1	کم‌ارزش‌ترین بیت کد
125–145	D_1	0	—
145–165	D_2	0	—
165–185	D_3	1	—
185–205	D_4	1	—
205–225	D_5	0	—
225–245	D_6	1	پرارزش‌ترین بیت کد
245–265	خاتمه	1	قرارداد قاب؛ بازگشت خط به حالت بی‌کار

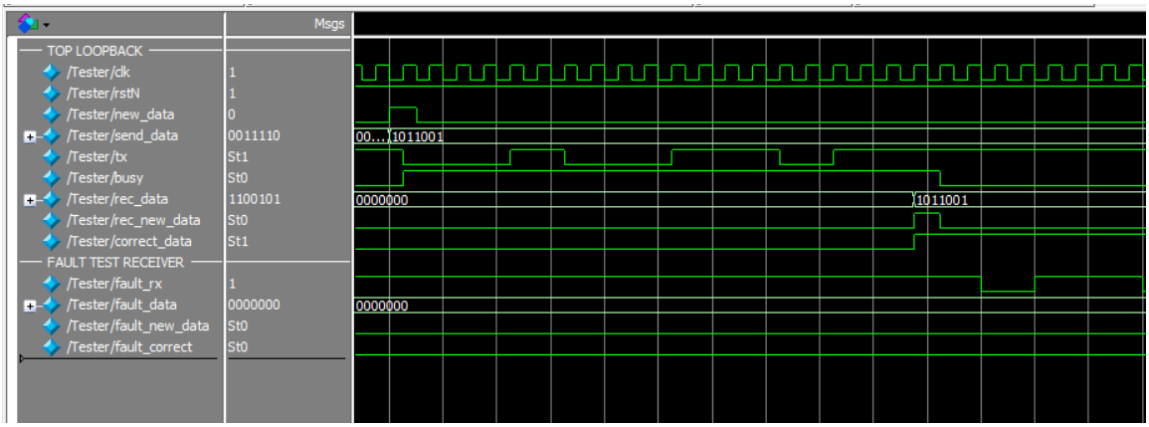
جدول ۴: تحلیل بازه‌به‌بازه‌ی قاب سالم مرحله‌ی نخست

اکنون می‌توان به پرسش اصلی پاسخ داد: چرا شکل موج tx در این بازه همان است که دیده می‌شود؟ اگر بیت‌های هم‌مقدار پشت سر هم را با هم ادغام کنیم، خط tx تنها در شش لحظه تغییر حالت می‌دهد و نه ده بار. جدول زیر همین را نشان می‌دهد و دقیقاً با لحظه‌های تغییری که در شکل موج دیده می‌شوند می‌خواند.

بازه (نانوثانیه)	مقدار	پهنا	ترکیب بیت‌ها
65–105	0	40	شروع و توازن، هر دو صفر
105–125	1	20	تنها D_0
125–165	0	40	D_1 و D_2، هر دو صفر
165–205	1	40	D_3 و D_4، هر دو یک
205–225	0	20	تنها D_5
225–265	1	40	D_6 و بیت خاتمه، هر دو یک

جدول ۵: پالس‌های دیده‌شده روی خط tx و ترکیب بیتی هر یک

پس آنچه در نما به‌صورت یک پالس پهن دیده می‌شود، دو بیت جداگانه با مقدار یکسان است و نه یک بیت طولانی. توجّه به همین نکته، تفاوت میان خواندن درست و نادرست شکل موج UART است.



شکل ۶: نمای بزرگ‌شده‌ی مرحله‌ی نخست؛ قاب سالم روی خط tx و بازسازی آن در گیرنده

د) لحظه‌های نمونه‌برداری گیرنده

تا اینجا رفتار فرستنده توضیح داده شد. برای اینکه روشن شود چرا خروجی گیرنده دقیقاً همان مقدار ارسالی می‌شود، باید دید گیرنده در چه لحظه‌هایی خط را می‌خواند. گیرنده در لحظهٔ ۷۵ نانوثانیه، یعنی نخستین لبهٔ فعّال پس از پایین رفتن خط، شروع را آشکار می‌کند و از همان‌جا شمارش خود را آغاز می‌نماید. جدول زیر لحظهٔ خواندن هر بیت را می‌آورد.

لحظه (نانوثانیه)	چه خوانده می‌شود	بازه‌ی آن بیت	مقدار
75	آشکارسازی شروع	85–65	0
95	بیت توازن	105–85	0
115	D_0	125–105	1
135	D_1	145–125	0
155	D_2	165–145	0
175	D_3	185–165	1
195	D_4	205–185	1
215	D_5	225–205	0
235	D_6	245–225	1
255	بیت خاتمه	265–245	1

جدول ۶: لحظه‌های نمونه‌برداری گیرنده و بازه‌ی متناظر هر بیت

ستون سوم این جدول، مهم‌ترین نکتهٔ زمان‌بندی طرح را آشکار می‌کند: هر لحظهٔ نمونه‌برداری دقیقاً ۱۰ نانوثانیه پس از آغاز بازهٔ بیت خود قرار دارد، یعنی درست در میانهٔ آن بازهٔ ۲۰ نانوثانیه‌ای. بنابراین در هر سو ۱۰ نانوثانیه فاصله تا لبه‌های بیت باقی می‌ماند و همین فاصله است که خواندن را در برابر اختلاف جزئی ساعت دو سوی خط ایمن می‌کند. اگر نمونه‌برداری نزدیک لبه انجام می‌شد، کوچک‌ترین ناهم‌خوانی می‌توانست بیت همسایه را به‌جای بیت درست بخواند.

ه) چرا خروجی گیرنده در لحظه‌ی ۲۵۵ نانوثانیه ظاهر می‌شود

در شکل موج دیده می‌شود که rec_data تا لحظهٔ ۲۵۵ نانوثانیه مقدار پیشین خود را نگه می‌دارد و تنها در آن لحظه به 1011001 تغییر می‌کند؛ هم‌زمان rec_new_data برای یک پالس ساعت بالا می‌رود و correct_data یک می‌شود. دلیل این زمان‌بندی از جدول بالا روشن است: گیرنده تا پیش از خواندن بیت خاتمه هیچ داوری‌ای نمی‌کند، چون تا آن لحظه هنوز نمی‌داند قاب سالم بوده است یا نه. در لحظهٔ ۲۵۵ نانوثانیه سه کار هم‌زمان انجام می‌شود:

- محتوای ثبات درونی به خروجی rec_data منتقل می‌شود.
- پرچم rec_new_data برای یک پالس بالا می‌رود تا پایان یک قاب کامل اعلام شود.
- دو شرط سلامت با هم سنجیده می‌شوند: بیت خاتمهٔ خوانده‌شده باید یک باشد و توازن محاسبه‌شده از روی هفت بیت ذخیره‌شده باید با بیت توازن دریافتی بخواند. در این قاب هر دو برقرارند، پس correct_data یک می‌شود.

از اینجا تأخیر کلّ مسیر نیز به دست می‌آید: از لحظهٔ ۶۵ که فرستنده درخواست را پذیرفت تا لحظهٔ ۲۵۵ که داده در خروجی گیرنده حاضر شد، ۱۹۰ نانوثانیه گذشته است. این عدد ده نانوثانیه کمتر از طول کامل قاب است، چون خروجی در لحظهٔ *نمونه‌برداری* بیت خاتمه به‌روز می‌شود و نه در پایان آن بیت.
پرچم busy نیز رفتار متناظری دارد: از ۶۵ تا ۲۶۵ نانوثانیه بالا می‌ماند، یعنی دقیقاً ۲۰۰ نانوثانیه که برابر طول کامل قاب است. تفاوت ۱۰ نانوثانیه‌ای میان لحظهٔ پایین آمدن busy و لحظهٔ ظاهر شدن خروجی گیرنده، همان اختلافی است که در بند پیش توضیح داده شد.

و) قاب معیوب توازن

در مرحلهٔ دوم، میز آزمون همان کد 1011001_2 را می‌فرستد امّا بیت توازن را عمداً وارونه می‌کند؛ یعنی به‌جای مقدار درست صفر، مقدار یک روی خط می‌گذارد. جدول زیر بازه‌های این قاب را نشان می‌دهد.

بازه (نانوثانیه)	بیت	fault_rx	وضعیّت
280–300	شروع	0	درست
300–320	توازن	1	**وارونه‌ی مقدار درست**
320–460	D_0 تا D_6	1001101	درست
460–480	خاتمه	1	درست

جدول ۷: بازه‌های قاب معیوب توازن در مرحله‌ی دوم

گیرنده این قاب را بی‌هیچ مشکلی تا انتها دریافت می‌کند و در لحظهٔ ۴۶۵ نانوثانیه پالس fault_new_data را می‌دهد؛ یعنی از دید ساختار قاب، همه‌چیز عادی بوده است. امّا در همان لحظه، توازن محاسبه‌شده از روی

هفت بیت داده صفر به دست می‌آید و با بیت توازن دریافتی که یک است نمی‌خواند، پس fault_correct صفر می‌ماند. این دقیقاً همان رفتاری است که انتظار می‌رفت: قاب کامل شد، ولی سالم نبود.

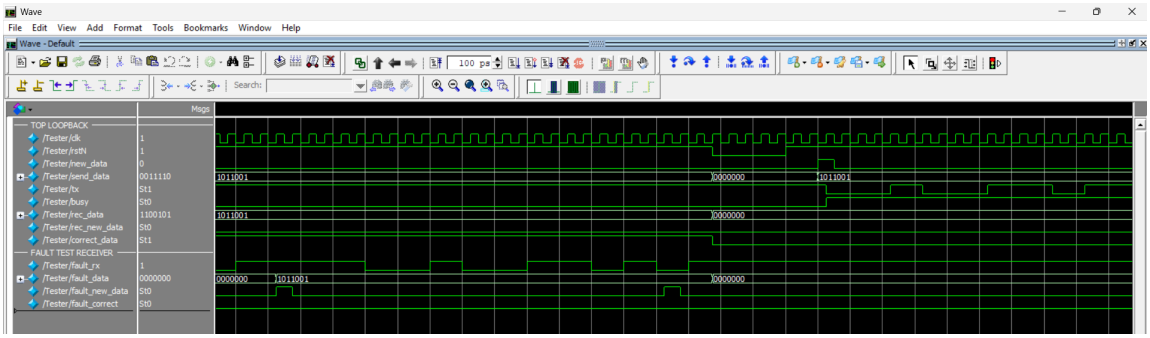
ز) قاب معیوب بیت خاتمه

در مرحلهٔ سوم، بیت توازن درست است ولی بیت خاتمه به‌جای یک، صفر گذاشته شده است.

بازه (نانوثانیه)	بیت	fault_rx	وضعیّت
520–540	شروع	0	درست
540–560	توازن	0	درست
560–700	D_0 تا D_6	1001101	درست
700–720	خاتمه	0	**باید یک می‌بود**

جدول ۸: بازه‌های قاب معیوب بیت خاتمه در مرحله‌ی سوم

گیرنده در لحظهٔ ۷۰۵ نانوثانیه بیت خاتمه را می‌خواند و مقدار صفر می‌بیند. چون یکی از دو شرط سلامت نقض شده است، fault_correct دوباره صفر می‌ماند، هرچند این بار توازن کاملاً درست بوده است. مقایسهٔ این دو مرحله نشان می‌دهد که هر یک از دو شرط به‌تنهایی می‌تواند قاب را نامعتبر کند و پرچم سلامت، یای منطقی آن دو نیست بلکه واو منطقی آن‌هاست.



شکل ۷: نمای بزرگ‌شده‌ی قاب‌های معیوب و پایین ماندن پرچم درستی

ح) بازه‌ی اشغال بودن فرستنده

مرحلهٔ پنجم، رفتار فرستنده را در برابر درخواستی می‌سنجد که در میانهٔ ارسال می‌رسد. دنبالهٔ رویدادها چنین است:

لحظه (نانوثانیه)	رویداد
1220	درخواست نخست با کد 1100101 اعمال می‌شود
1225	فرستنده درخواست را می‌پذیرد و busy بالا می‌رود
1260	درخواست دوم با کد 0011110 در میانه‌ی ارسال اعمال می‌شود
1415	قاب نخست کامل می‌شود؛ rec_data برابر 1100101
1425	busy پایین می‌رود و فرستنده آزاد می‌شود

جدول ۹: دنباله‌ی رویدادهای مرحله‌ی پنجم

در لحظهٔ ۱۲۶۰ نانوثانیه، ماشین فرستنده در میانهٔ حالت داده است و نه در حالت بی‌کار. چون در کد، پذیرش درخواست تنها در حالت بی‌کار بررسی می‌شود، این درخواست هیچ اثری نمی‌گذارد: نه ثبات دادهٔ درونی بازنویسی می‌شود و نه ماشین حالت از مسیر خود منحرف می‌گردد. شاهد این ادّعا در شکل موج آن است که با وجود تغییر send_data به 0011110 در لحظهٔ ۱۲۶۰، مقدار بازسازی‌شده در گیرنده در لحظهٔ ۱۴۱۵ همچنان 1100101 است؛ یعنی همان کدی که پیش از اشغال شدن فرستنده قفل شده بود.
میز آزمون یک گام فراتر می‌رود و پس از آزاد شدن فرستنده، به مدّت ده طول بیت دیگر خط را می‌پاید تا مطمئن شود درخواست نادیده‌گرفته‌شده بعداً هم قاب اضافه‌ای نمی‌سازد. در این بازه هیچ پالس تازه‌ای روی rec_new_data ظاهر نمی‌شود، پس درخواست به‌کلّی دور ریخته شده و صرفاً به تعویق نیفتاده است.

ط) جمع‌بندی تحلیل

آنچه از بررسی این بازه‌ها به دست می‌آید چنین است:

- هر بیت روی خط دقیقاً ۲۰ نانوثانیه و هر قاب دقیقاً ۲۰۰ نانوثانیه طول می‌کشد و همهٔ لحظه‌های تغییر، مضرب صحیحی از این عددند.
- پهنای نایکسان پالس‌های tx نه نشانهٔ بی‌نظمی، بلکه پیامد ادغام دیداری بیت‌های هم‌مقدار پیاپی است.
- گیرنده هر بیت را در میانهٔ بازهٔ آن و با ۱۰ نانوثانیه حاشیه از هر دو لبه می‌خواند.
- خروجی گیرنده تنها در لحظهٔ نمونه‌برداری بیت خاتمه به‌روز می‌شود، که تأخیر کلّ مسیر را ۱۹۰ نانوثانیه می‌کند.
- پرچم پایان قاب و پرچم سلامت قاب مستقل از یکدیگرند؛ در هر دو قاب معیوب، اوّلی بالا رفت و دوّمی پایین ماند.
- درخواست ارسال در بازهٔ اشغال بودن، نه قاب جاری را تغییر داد و نه بعداً قاب تازه‌ای ساخت.

۹.۷ سنتز و پیاده‌سازی نرم‌افزاری

الف) مصرف منابع

سه پروندهٔ سنتزپذیر طرح، بدون میز آزمون، در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شدند. انتخاب قطعه روی حالت خودکار گذاشته شد و ابزار قطعهٔ EP4CGX15BF14C6 را برگزید. نتیجهٔ همگردانی ۲۱۵ عنصر منطقی شامل ۲۰۹ تابع ترکیبی و ۱۰۶ ثبات اختصاصی، و ۲۱ پایه است.

ب) بررسی شمار ثبات‌ها

شمار ۱۰۶ ثبات برای مداری با این اندازه در نگاه نخست زیاد به نظر می‌رسد و ارزش بررسی دارد. گزارش سنتز، ۵۰ ثبات را به فرستنده و ۵۶ ثبات را به گیرنده نسبت می‌دهد. اگر ثبات‌های ساختاری هر پیمانه شمرده شوند، به این اعداد می‌رسیم:

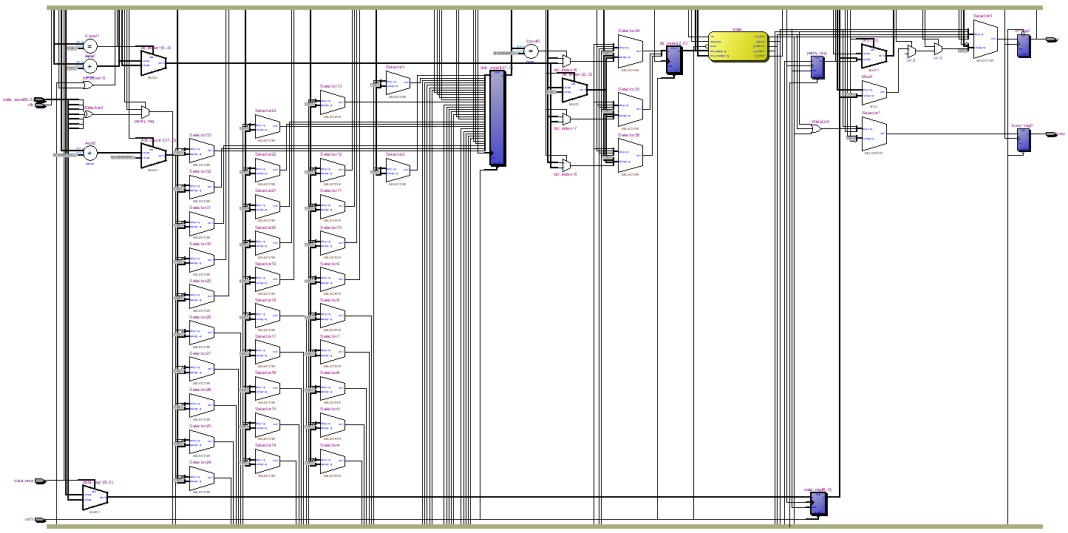
- فرستنده: پنج بیت برای ثبات حالت با رمزگذاری یک‌داغ[۱۴]، سه بیت bit_index، هفت بیت دادهٔ ذخیره‌شده، یک بیت توازن، و دو بیت برای خروجی‌های tx و busy؛ در مجموع هجده بیت.
- گیرنده: چهار بیت حالت، سه بیت bit_index، هفت بیت ثبات داخلی، یک بیت توازن دریافتی، هفت بیت خروجی داده و دو بیت برای دو پرچم؛ در مجموع بیست و چهار بیت.

اختلاف این اعداد با گزارش، در هر دو پیمانه دقیقاً سی و دو بیت است. منشأ آن روشن است: شمارندهٔ tick_count در هر دو پیمانه از نوع integer تعریف شده و این نوع در Verilog همواره سی و دو بیت است. بنابراین دو شمارنده که برای مقدار پیش‌فرض TICKS_BIT تنها به یک بیت نیاز دارند، در عمل شصت و چهار فلیپ‌فلاپ گرفته‌اند؛ یعنی نزدیک به شصت درصد کلّ ثبات‌های طرح.
این نکته بر درستی عملکرد اثری ندارد و در شبیه‌سازی نیز دیده نمی‌شود، امّا از نظر مصرف منابع بهبودپذیر است: اگر این شمارنده به‌جای integer با پهنای $\lceil \log_2(\text{TICKS_BIT}) \rceil$ بیت تعریف می‌شد، شمار ثبات‌های طرح از ۱۰۶ به حدود ۴۲ کاهش می‌یافت. آوردن این مشاهده در گزارش از آن روست که تفاوت میان «کد درست» و «کد بهینه» را نشان می‌دهد.

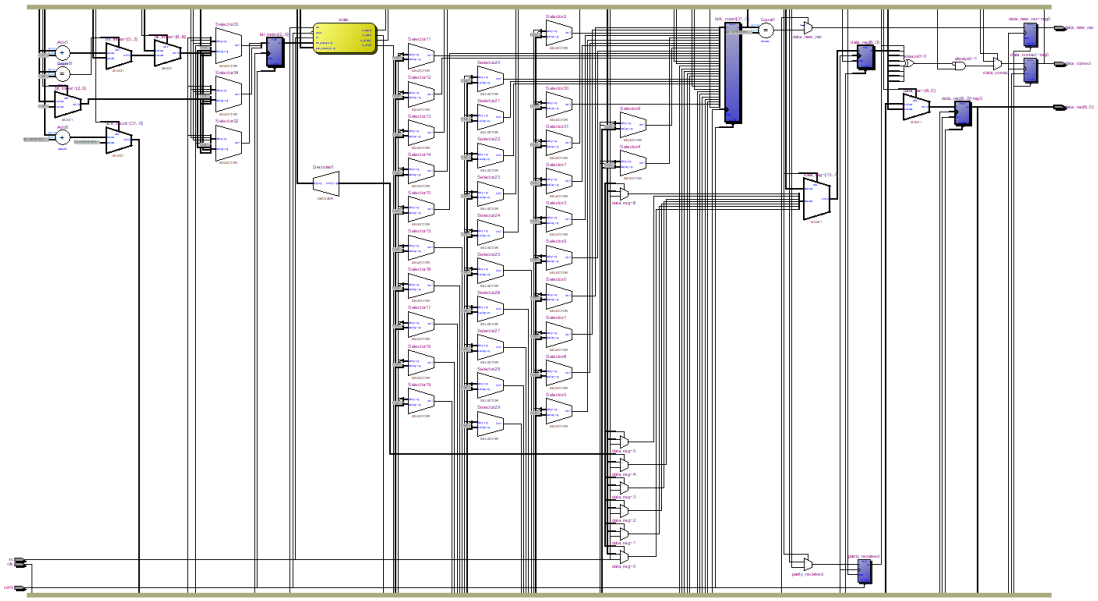
ج) نماهای سطح انتقال ثبات‌ها و فناوری

نمای سطح انتقال ثبات‌های فرستنده، ثبات حالت، شمارندهٔ بیت، ثبات دادهٔ ذخیره‌شده و منطق تولید بیت توازن را نشان می‌دهد. در نمای گیرنده نیز ثبات حالت، ثبات داده و منطق مقایسهٔ توازن دیده می‌شود.

[14] One-Hot Encoding

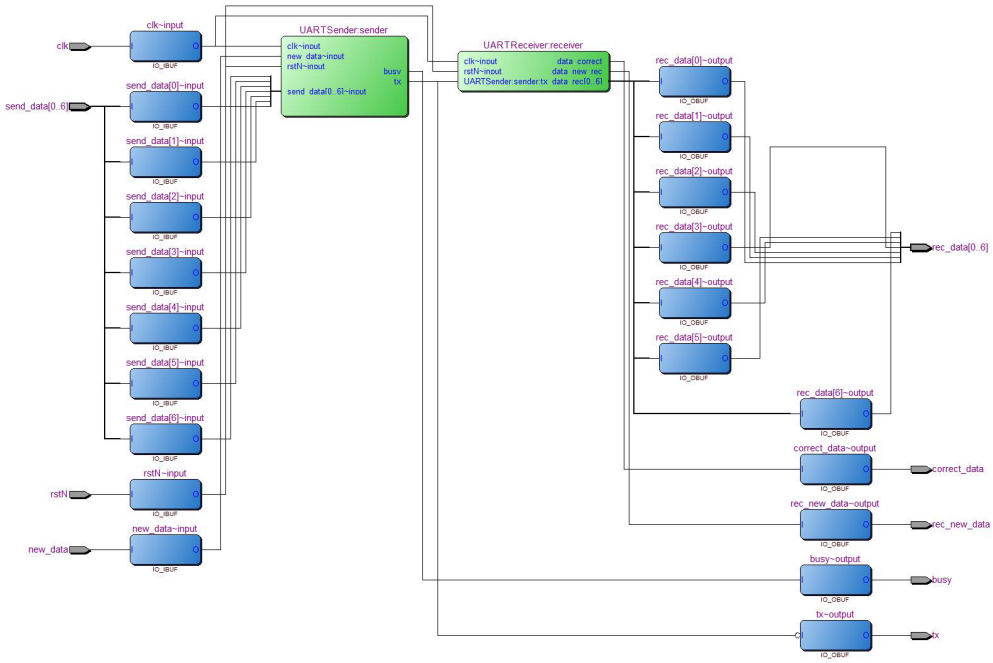


شکل ۸: نمای سطح انتقال ثبات‌های فرستنده

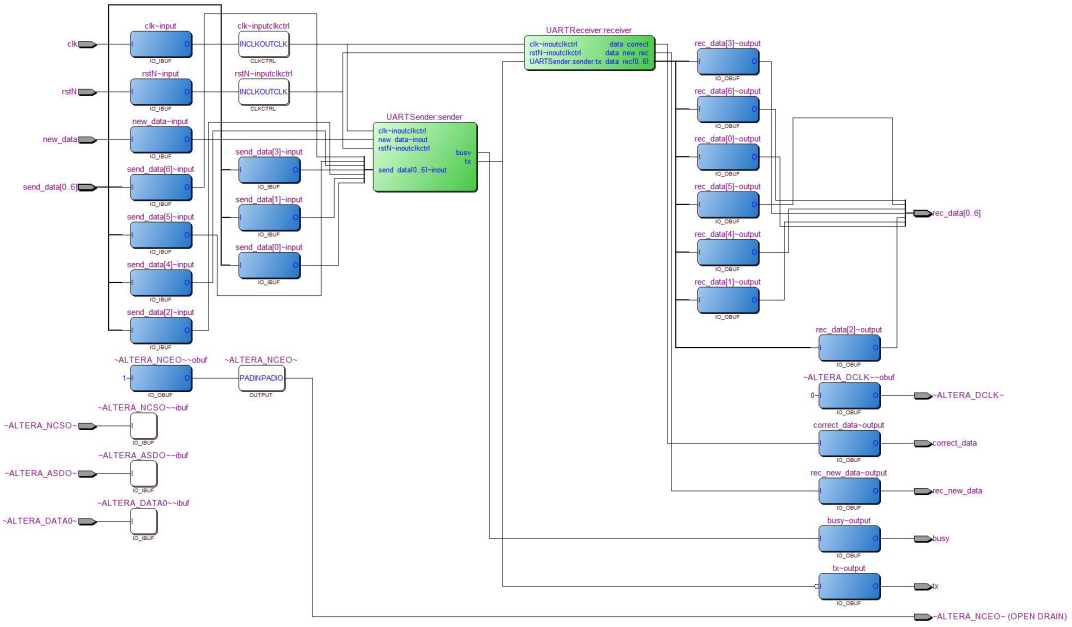


شکل ۹: نمای سطح انتقال ثبات‌های گیرنده

نماهای فناوری پس از نگاشت و پس از جانمایی نشان می‌دهند که این ساختار به جدول‌های جست‌وجو و فلیپ‌فلاپ‌های واقعی قطعه نگاشت و مسیریابی شده است. نماهای سطح بالا در زیر آمده‌اند؛ نماهای درونی هر یک از دو پیمانه نیز ثبت شده‌اند، امّا چون بسیار پهن‌اند و در عرض صفحه خوانا نمی‌شوند، تنها در بستهٔ تحویلی آورده شده‌اند.



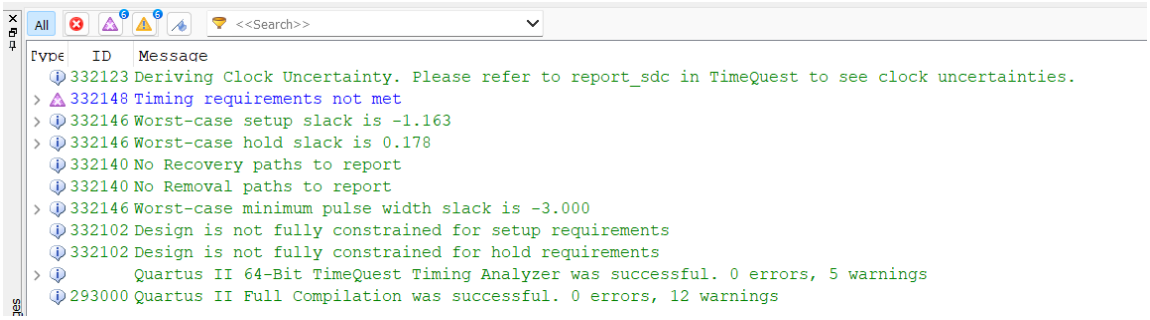
شکل ۱۰: نمای Technology Map Viewer (Post-Mapping) در سطح بالا



شکل ۱۱: نمای Technology Map Viewer (Post-Fitting) در سطح بالا

د) تحلیل زمانی

برای این پروژه پروندهٔ قیود طراحی[۱۵] تعریف نشده است؛ در چنین حالتی تحلیل‌گر زمانی[۱۶] به‌طور خودکار ساعتی با دورهٔ یک نانوثانیه، معادل یک گیگاهرتز، استنتاج می‌کند. طبیعی است که طرح در برابر چنین قید تحمیلی و غیرواقع‌بینانه‌ای حاشیهٔ زمانی[۱۷] منفی نشان دهد و هشدار Timing requirements not met تولید شود.



شکل ۱۲: پیام‌های پایانی همگردانی و هشدارهای تحلیل زمانی

اعداد گزارش‌شده چنین‌اند: بدترین حاشیهٔ برپایی در مدل تند -1.163 و در مدل کند -2.892 نانوثانیه، و حاشیهٔ نگهداری در هر دو مدل مثبت. همگردانی کامل با صفر خطا به پایان رسید.

معیار درست ارزیابی در نبود قید، بیشینه‌بسامد[۱۸] گزارش‌شده است. تحلیل‌گر زمانی برای بدترین شرایط کاری مقدار $256.94\,\text{MHz}$ را گزارش می‌کند و بیشینه‌بسامد مقیّد[۱۹] برابر $250\,\text{MHz}$ است که از محدودیّت نرخ تغییر پایه‌های ورودی و خروجی[۲۰] قطعه می‌آید و نه از منطق درونی طرح. ساعت ۱۰۰ مگاهرتزی که در میز آزمون به کار رفته، بسیار پایین‌تر از این حدّ است.

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۱۰.۷ نتایج

- قاب ده‌بیتی با ترتیب خواسته‌شدهٔ دستورکار، یعنی شروع، توازن، هفت بیت داده و خاتمه، در فرستنده پیاده و در گیرنده بازخوانی شد.

[15] Synopsys Design Constraints
[16] TimeQuest Timing Analyzer
[17] Slack
[18] Maximum Frequency (Fmax)
[19] Restricted Fmax
[20] Maximum I/O Toggle Rate

- فرستنده و گیرنده هر یک به‌صورت یک ماشین حالت جداگانه نوشته شدند و پیمانهٔ بالا با اتّصال حلقهٔ بازگشتی، مبادلهٔ داده میان آن دو را نشان داد.
- طول هر بیت با یک پارامتر بیان شد تا طرح به نرخ باود مشخّصی وابسته نماند.
- افزون بر ذخیرهٔ داده، گیرنده سلامت قاب را نیز گزارش می‌کند و هر دو گونهٔ خطای توازن و خطای بیت خاتمه را تشخیص داد.
- هر پنج مرحلهٔ میز آزمون خودآزما، شامل دو قاب پیاپی و درخواست هم‌زمان با اشغال، بدون خطا گذشتند.
- سنتز با ۲۱۵ عنصر منطقی، ۱۰۶ ثبات و ۲۱ پایه انجام شد و بررسی نشان داد شصت و چهار ثبات از این شمار، تنها به دلیل تعریف شمارنده‌ها از نوع integer مصرف شده‌اند.
- بیشینه‌بسامد گزارش‌شده $256.94\,\mathrm{MHz}$ با حدّ مقیّد $250\,\mathrm{MHz}$ است.

نتیجه‌گیری

در این آزمایش یک واحد UART کامل با فرستنده و گیرندهٔ جداگانه ساخته و درستی مبادلهٔ داده میان آن دو نشان داده شد. آنچه این آزمایش را از یک تمرین ماشین حالت ساده متمایز می‌کند، ناهم‌گام بودن پیوند دو سوی خط است: هیچ ساعت مشترکی میان فرستنده و گیرنده رد و بدل نمی‌شود و تنها قرارداد مشترک، ساختار قاب و طول هر بیت است. به همین دلیل، نمونه‌برداری در میانهٔ هر بیت و نه در لبه‌های آن انجام شد. افزودن وارسی توازن و بیت خاتمه نیز نشان داد که یک قاب کامل‌شده لزوماً یک قاب سالم نیست و این دو مفهوم باید در دو پرچم جداگانه گزارش شوند. سرانجام، بررسی گزارش سنتز یادآور شد که درستی کارکردی و کارآمدی سخت‌افزاری دو چیز متفاوت‌اند؛ چنان‌که در همین طرحِ کاملاً درست، بیش از نیمی از فلیپ‌فلاپ‌ها تنها به سبب یک انتخاب نوع داده مصرف شده‌اند.